Cletus: Week 1, Day 5

> To copy code: select it here, or click a "Copy on GitHub" link and use the copy button there. Full note online: https://github.com/mangucletus/AI-Engineering-Course/blob/main/week-1/cletus-week1-day5.md

- Date: 2026-10-09 (Friday)
- Topic: brochure generator, JSON output, chaining calls, streaming
- Notebook: week1/day5.ipynb, week1/week1 EXERCISE.ipynb

Key concepts

- Chaining: call 1 picks the relevant links (JSON), call 2 writes the brochure from those pages. First taste of agents.
- One-shot prompt: show one example of the output you want.
- JSON mode: response_format={"type": "json_object"}, then json.loads(). The prompt must mention JSON.
- Streaming: stream=True, then read chunk.choices[0].delta.content.
- Tone is set by the system prompt.
- Truncate long input to save tokens.

Code / examples

JSON output:

Copy on GitHub

```
import json
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv(override=True)
openai = OpenAI()

system = 'Reply in JSON like {"links": [{"type": "about page", "url": "https://..."}]}'
user = "Pick the About and Careers links: /about, /careers, /privacy, /blog"
response = openai.chat.completions.create(
    model="gpt-5-nano",
    messages=[{"role": "system", "content": system}, {"role": "user", "content": user}],
    response_format={"type": "json_object"},
)
print(json.loads(response.choices[0].message.content))
```

Streaming:

Copy on GitHub

```
stream = openai.chat.completions.create(
    model="gpt-4.1-mini",
    messages=[{"role": "user", "content": "Write a 3-line brochure for a coffee shop"}],
    stream=True,
)
for chunk in stream:
    print(chunk.choices[0].delta.content or "", end="")
```

Exercise / challenge

Week 1 challenge: technical tutor. Same question to GPT and Llama 3.2, compare.

Copy on GitHub

```
question = "Explain: yield from {b.get('author') for b in books if b.get('author')}"
messages = [
    {"role": "system", "content": "You are a helpful technical tutor."},
    {"role": "user", "content": question},
]
ollama = OpenAI(base_url="http://localhost:11434/v1", api_key="ollama")

for client, model in [(openai, "gpt-4o-mini"), (ollama, "llama3.2")]:
    response = client.chat.completions.create(model=model, messages=messages)
    print(f"--- {model} ---\n{response.choices[0].message.content}\n")
```

Questions & blockers

- Some links fail to scrape. Relative links (/about) need the full URL.

Resources

- week1/scraper.py and week1/solutions/ in the course repo